Machine Learning based on Aitchison Geometry for Hyperspectral Imaging

ANANDARAJAH Rahul, supervised by ANDERSEN Ang

January 31, 2025

Contents

1. Context of the problem **3**
1.1. Definition (Probability vector) . . . 3
1.2. Definition (Problem formulation) . . . 3
1.3. Definition (Gradient Descent Algorithm) . . . 3
1.4. Lemma . . . 3
1.5. Proof Lemma 1.4. . . . 4
1.6. Lemma . . . 4
1.7. Proof Lemma1.5. . . . 4
1.8. Proof 1.3. (Gradient Descent Algorithm Convergence) . . . 5
1.9. Conclusion . . . 5

2. Aitchison algebra **6**
2.1. Definition (Aitchison closure) . . . 6
2.2. Definition (Aitchison perturbation) . . . 6
2.3. Definition (Aitchison scalar multiplication) . . . 6
2.4. Definition (Aitchison inverse) . . . 6
2.5. Definition (Aitchison inverse perturbation) . . . 6
2.6. Definition (Aitchison clr transformation) . . . 6
2.7. Definition (Aitchison inverse clr transformation) . . . 6
2.8. Interpretation of clr . . . 7
2.9. Definition (Aitchison inner product) . . . 7
2.10. Remark (clr isometry) . . . 7
2.11. Definition (perturbation independence) . . . 8
2.12. Definition (Orthonomal basis of S^D) . . . 8
2.13. Property (Orthonomal basis of S^D) . . . 8
2.14. Definition (Aitchison Norm) . . . 9
2.15. Definition (Aitchison Distance) . . . 9

3. Implementation and benchmark in Python **11**
3.1. Usual GD in euclidean space . . . 11
3.2. GD in Aitchison algebra . . . 12

4. Failed attempts at implementing Aitchison GD **16**
4.1. Approach 1 . . . 16
4.2. Approach 2 . . . 16
4.3. Approach 3 . . . 16
4.3.1. Definition (Euclidean basis decomposition and Euclidean directional derivative) 16
4.3.2. Definition (Euclidean Gradient) . . . 17
4.3.3. Definition (Aitchison basis decomposition and Aitchison directional derivative) . 17
4.3.4. Definition (Aitchison Gradient) . . . 17

4.4. Heatmap on ternary diagram 17

5. Conclusion and research perspectives **18**
5.1. Additional functions for testing Aitchison GD and Aitchison Least Squares 18
5.2. Image processing 18

6. References **19**

1. Context of the problem

1.1. Definition (Probability vector)

Let us define the set of probability vectors. For all $D \in \mathbb{N}^*$:

$$S^D = \left\{ x = (x_1, \ldots, x_D)^T \in \mathbb{R}^D : \sum_{i=1}^{D} x_i = 1, \quad x_i \geq 0 \quad \forall i \in [\![1, D]\!] \right\} \tag{1}$$

In Machine Learning, we often work with probability vectors to model probability densities. Working with this simplex is therefore very interesting.

1.2. Definition (Problem formulation)

We aim to find the minimum of a function $f : \mathbb{R}^D \longrightarrow \mathbb{R}$ over S^D.

$$\min_{x \in S^D} f(x) \tag{2}$$

1.3. Definition (Gradient Descent Algorithm)

We consider a simplified case of (2) where we minimize the function on $\mathbb{R}^D$ instead of S^D. We propose the following algorithm. Let $(x_k)_k$ denote the Gradient Descent algorithm (GD) :

$$x_{k+1} = x_k - \gamma_k \nabla f(x_k) \tag{3}$$

where k is the iteration index, x_k is the value of the algorithm at step k, and γ_k is the step size at step k.

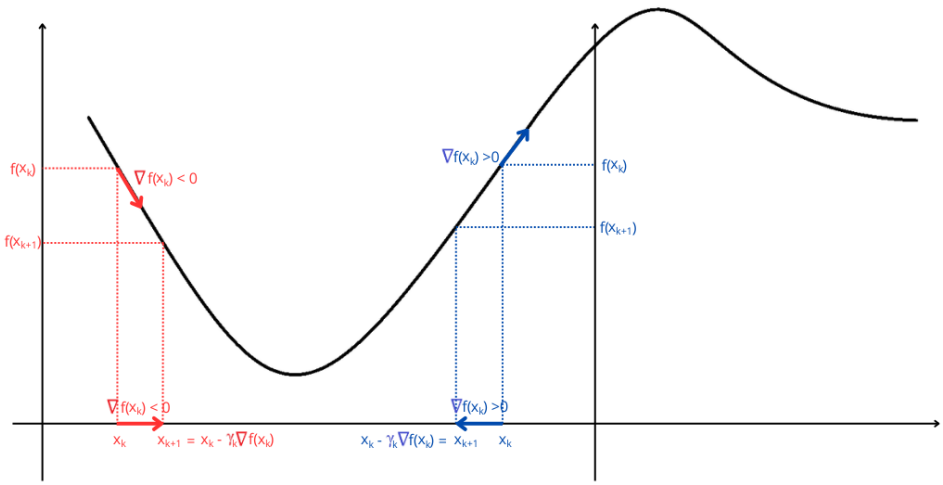

Figure 1: If x_k lies in a convex region of the function, the algorithm converges to a local minimum. If f is convex, the algorithm converges to the global minimum.

To prove the convergence of the algorithm, we need to demonstrate Lemmas 1.4 and 1.5.

1.4. Lemma

Let $A \in S_D(\mathbb{R})$. Then :

$$\|A\| \leq \max |\mathrm{Sp}(A)| \tag{4}$$

1.5. Proof Lemma 1.4.

Let A be a real symmetric matrix. By the spectral theorem, A is diagonalizable in an orthonormal basis $\mathcal{B}$ of eigenvectors. There exist $P \in \mathcal{GL}_D(\mathbb{R})$ and $B \in \mathcal{D}_D(\mathbb{R})$ such that :

$$A = PBP^{-1}, \quad \text{with } B = \operatorname{diag}(\lambda_1, \dots, \lambda_D) \quad \text{and } \operatorname{Sp}(A) = \{\lambda_1, \dots, \lambda_D\} \tag{5}$$

Let $u = (u_1, \dots, u_D)^T$ be decomposed in the basis $\mathcal{B}$. Then $Bu = (\lambda_1 u_1, \dots, \lambda_D u_D)^T$. Then :

$$\|Bu\|^2 = \sum_{i=1}^{D} (\lambda_i u_i)^2 \leq \lambda_{\max}^2 \sum_{i=1}^{D} u_i^2 = \lambda_{\max}^2 \|u\|^2 . \tag{6}$$

Thus :

$$\frac{\|Bu\|}{\|u\|} \leq |\lambda_{\max}| \tag{7}$$

Taking the supremum :

$$\sup_{u \in \mathbb{R}^D} \frac{\|Bu\|}{\|u\|} \leq |\lambda_{\max}| \tag{8}$$

Since $\sup_{u \in \mathbb{R}^D} \frac{\|Bu\|}{\|u\|}$ defines a norm on $\mathbb{R}^D$, by the equivalence of norms in finite dimensions and knowing that B and A are similar matrices (and thus have the same norm) :

$$\|A\| \leq |\lambda_{\max}| \tag{9}$$

1.6. Lemma

Let $f : \mathbb{R}^D \longrightarrow \mathbb{R}$ such that ∇f is L-Lipschitz continuous. Let $\lambda_{\max}$ denote the largest eigenvalue of $\mathcal{H}_f$ the Hessian matrix of f in absolute value. Then :

$$\lambda_{\max} \leq L \tag{10}$$

1.7. Proof Lemma1.5.

Let $(x, y) \in (\mathbb{R}^D)^2$, from Taylor's formula :

$$\nabla f(y) = \nabla f(x) + \mathcal{H}_f(y - x) + o(\|y - x\|) \tag{11}$$

Since ∇f is L-Lipschitz, we have :

$$\|\nabla f(y) - \nabla f(x)\| = \|\mathcal{H}_f(y - x) + o(\|y - x\|)\| \leq L \|y - x\| \tag{12}$$

Let's choose $(x, y) \in (\mathbb{R}^D)^2$ such that $y - x = te_{\max}$, where $t \in \mathbb{R}$ and $e_{\max}$ satisfies $\mathcal{H}_f e_{\max} = \lambda_{\max} e_{\max}$ with $\|e_{\max}\| = 1$. Then :

$$\|t\lambda_{\max} e_{\max} + o(t \|e_{\max}\|)\| \leq L \|te_{\max}\| \tag{13}$$

Thus :

$$|\lambda_{\max}| \leq L \tag{14}$$

1.8. Proof 1.3. (Gradient Descent Algorithm Convergence)

Let us show that the GD algorithm converges. To do so, we must make some assumptions about f. Specifically, ∇f must be L-Lipschitz continuous, and $\gamma_k \in [0, \frac{1}{L}]$. We will assume that γ_k is constant and less than or equal to $\frac{1}{L}$. Let $(x, y) \in (\mathbb{R}^D)^2$:

$$f(y) = f(x) + \langle \nabla f(x), y - x \rangle + \frac{1}{2}(y - x)^T \mathcal{H}_f(y_{1/2})(y - x) \tag{15}$$

where $y_{1/2}$ is some point between x and y.

From lemmas 1 and 2 and triangular inequality, we have :

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2 \tag{16}$$

By choosing two consecutive terms of the algorithm, that is, for every $n \in \mathbb{N}$, let $y = x_{n+1}$ and $x = x_n$, we obtain :

$$f(x_{n+1}) \leq f(x_n) - \gamma_n \|\nabla f(x_n)\|^2 + \frac{L}{2}\gamma_n^2 \|\nabla f(x_n)\|^2 \tag{17}$$

By choosing γ_n such that $\gamma_n \leq \frac{1}{L}$, we get :

$$f(x_n) - f(x_{n+1}) \geq \frac{1}{2L}\|\nabla f(x_n)\|^2 \tag{18}$$

By telescoping :

$$\sum_{i=0}^{D} L\|\nabla f(x_i)\|^2 \leq 2\left(f(x_0) - f(x_{D+1})\right) \tag{19}$$

Finally :

$$\min_{0 \leq i \leq D} \|\nabla f(x_i)\|^2 \leq \frac{2\left(f(x_0) - f^*\right)}{(D+1)L} \tag{20}$$

Thus, the left-hand side tends to 0 as $D \to \infty$.

1.9. Conclusion

The algorithm works on $\mathbb{R}^D$. There is no reason it should solve (2). To address this issue and stay within S^D, we do the following :

$$x_{k+1} = \mathrm{proj}_{S^D}(x_k - \gamma_k \nabla f(x_k)) \tag{21}$$

However, projecting every term at each iteration of the GD algorithm involves computational costs of $O(n \log n)$ complexity. <u>How can we reduce these costs ?</u>

2. Aitchison algebra

Let us define an algebra on S^D. For any <u>row</u> vector $x \in \mathbb{R}^D \setminus \{0\}$, we denote $x = [x_1, \dots, x_D]$, with the geometric mean $g(x) = \sqrt[D]{\prod_{i=1}^D x_i}$. For each of the definition introduced in this section, we will first provide a definition in Aitchison's sense, followed by its rewriting using Euclidean tools.

2.1. Definition (Aitchison closure)

$$\mathcal{C} : \begin{cases} \mathbb{R}_+^D \setminus \{0\} & \longrightarrow S^D \\ x & \longmapsto \left[\frac{x_1}{\sum_{i=1}^D x_i}, \dots, \frac{x_D}{\sum_{i=1}^D x_i}\right] = \frac{x}{\|x\|_1} \end{cases} \tag{22}$$

2.2. Definition (Aitchison perturbation)

For all $(x, y) \in (S^D \setminus \{0\})^2$:

$$x \oplus y = \mathcal{C}([x_1 y_1, \dots, x_D y_D]) \tag{23}$$

$$= \left[\frac{x_1 y_1}{\sum_{i=1}^D x_i y_i}, \dots, \frac{x_D y_D}{\sum_{i=1}^D x_i y_i}\right] \tag{24}$$

2.3. Definition (Aitchison scalar multiplication)

For all $(\alpha, x) \in \mathbb{R} \times S^D \setminus \{0\}$:

$$\alpha \odot x = \mathcal{C}([x_1^\alpha, \dots, x_D^\alpha]) = \mathcal{C}(x^{\odot\alpha}) \tag{25}$$

$$= \left[\frac{x_1^\alpha}{\sum_{i=1}^D x_i^\alpha}, \dots, \frac{x_D^\alpha}{\sum_{i=1}^D x_i^\alpha}\right] \tag{26}$$

2.4. Definition (Aitchison inverse)

For all $x \in S^D \setminus \{0\}$:

$$x^{-1} = \mathcal{C}(x^{\odot -1}) = (-1) \odot x \tag{27}$$

2.5. Definition (Aitchison inverse perturbation)

For all $(x, y) \in (S^D \setminus \{0\})^2$:

$$x \ominus y = x \oplus y^{-1} \tag{28}$$

2.6. Definition (Aitchison clr transformation)

$$\mathrm{clr} : \begin{cases} S^D \setminus \{0\} & \longrightarrow \mathbb{R}^D \\ x & \longmapsto \left[\log\left(\frac{x_1}{g(x)}\right), \dots, \log\left(\frac{x_D}{g(x)}\right)\right] \end{cases} \tag{29}$$

2.7. Definition (Aitchison inverse clr transformation)

$$\mathrm{clr}^{-1} : \begin{cases} \mathbb{R}^D \setminus \{0\} & \longrightarrow S^D \\ z & \longmapsto \mathcal{C}\left([\exp(z_1), \dots, \exp(z_D)]\right) \\ & = \left[\frac{\exp(z_1)}{\sum_{i=1}^D \exp(z_i)}, \dots, \frac{\exp(z_D)}{\sum_{i=1}^D \exp(z_i)}\right] \end{cases} \tag{30}$$

2.8. Interpretation of clr

$\mathrm{clr} : S^D \to \mathbb{R}^D$ is a bijection between S^D and $\mathbb{R}^D$. In other words, each element $x \in S^D$ has a bijective representation $y \in \mathbb{R}^D$ and vice versa, such that : $x = \mathrm{clr}^{-1}(y) \quad \text{and} \quad y = \mathrm{clr}(x)$.

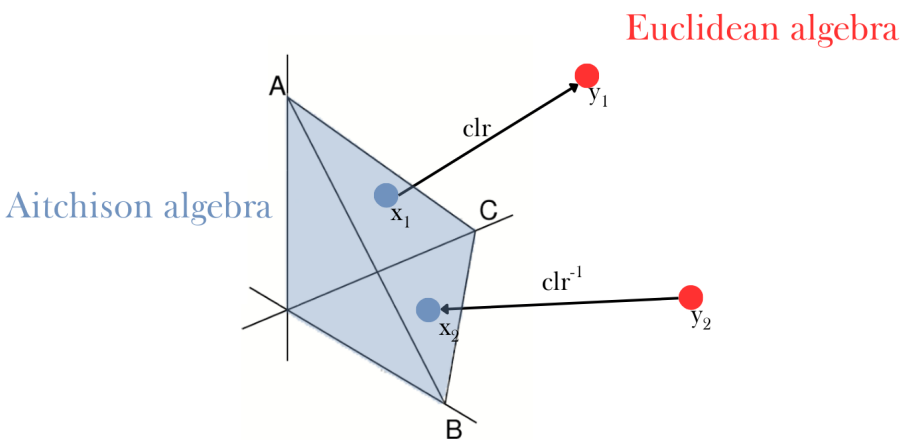

Figure 2: clr is a bijective mapping. For example, x_1 in the Aitchison algebra has a unique representation y_1 in the Euclidean algebra, just as y_2 in the Euclidean algebra has a unique representation x_2 in the Aitchison algebra.

2.9. Definition (Aitchison inner product)

For all $(x, y) \in (S^D \setminus \{0\})^2$:

$$\langle x, y\rangle_a = \langle \mathrm{clr}(x), \mathrm{clr}(y)\rangle_E \tag{31}$$

$$= \sum_{i<j}^{D} \ln x_i \ln x_j - \frac{1}{D}\left(\sum_{j=1}^{D} \ln x_j\right)\left(\sum_{k=1}^{D} \ln x_k\right) \tag{32}$$

$$= \frac{1}{D}\sum_{i=1}^{D-1}\sum_{j=i+1}^{D} \ln\frac{x_i}{x_j}\ln\frac{y_i}{y_j} \tag{33}$$

$$= \frac{1}{2D}\sum_{i=1}^{D}\sum_{j=1}^{D} \ln\frac{x_i}{x_j}\ln\frac{y_i}{y_j} \tag{34}$$

2.10. Remark (clr isometry)

The following property shows that clr is an isometry between S^D and $\mathcal{V}^D$:

$$\langle x, y\rangle_A = \langle \mathrm{clr}(x), \mathrm{clr}(y)\rangle_E \quad \text{with} \quad \mathcal{V}^D = \left\{ v \in \mathbb{R}^D \mid \sum_{i=1}^{D} v_i = 0 \right\} \tag{35}$$

Let's explain why we get the condition on $\mathcal{V}^D$:

$$\mathrm{clr}(x) = \left[\log\left(\frac{x_1}{g(x)}\right), \ldots, \log\left(\frac{x_D}{g(x)}\right)\right] \tag{36}$$

Thus,

$$\sum_{i=1}^{D} \mathrm{clr}(x)_i = \sum_{i=1}^{D} \log\left(\frac{x_i}{g(x)}\right) = \log\left(\frac{\prod_{i=1}^{D} x_i}{g(x)^D}\right) = 0 \tag{37}$$

2.11. Definition (perturbation independence)

The definition of linearly independent vectors in a generic vector space also applies to the Aitchison algebra. $(b_i)_{i\in[\![1,D-1]\!]} \in (S^D)^{D-1}$ is a set of independent vectors if and only if the following implication holds :

$$\left(\forall\,(\alpha_1,\dots,\alpha_{D-1}) \in \mathbb{R}^{D-1}, \quad \bigoplus_{i=1}^{D-1} \alpha_i \odot b_i = \mathbf{n}\right) \implies (\alpha_1 = \cdots = \alpha_{D-1} = 0) \tag{38}$$

where $\mathbf{n} = \mathcal{C}([1\dots1])$ is the neutral element of the Aitchison algebra.

2.12. Definition (Orthonomal basis of $\mathbf{S}^D$)

In order to find an Aitchison orthonormal basis $(b_i)_{i\in[\![1,D-1]\!]}$ of S^D in the Aitchison sense, one approach is to first determine an Euclidean orthonormal basis $(v_i)_{i\in[\![1,D-1]\!]}$ of $\mathcal{V}^D$, and then apply the clr^{-1} transform. How can we determine the expression of v_i ? Let V be a $D \times (D-1)$ matrix, where the columns v_i are expressed in the canonical basis of $\mathbb{R}^D$. Then :

$$\begin{cases} V^T V & = I_{D-1} \\ V V^T & = I_D - \mathbf{1}_D \mathbf{1}_D^T \end{cases} \tag{39}$$

The first condition ensures orthonormality, while the second condition, in accordance with (35), guarantees that the components sum to zero. There are multiple solutions to this problem, leading to different possible Aitchison algebras, as discussed during the meetings.

2.13. Algorithm (Find V)

The algorithm generates a matrix V of size $D \times (D-1)$, whose coefficients are defined by the formula :

$$V_{i,j} = \begin{cases} \frac{1}{\sqrt{(j+1)(j+2)}} & \text{if } i \leq j \\ -\sqrt{\frac{j+1}{j+2}} & \text{if } i = j+1 \\ 0 & \text{otherwise} \end{cases} \tag{40}$$

This matrix ensures that (39) is fulfilled. Then, we get the explicit expression of $(e_i)_{i\in[\![1,D-1]\!]}$ by using the conversion/bijection between Aitchison basis and Euclidean basis :

$$b_i = \mathrm{clr}^{-1}(v_i) \quad i = 1,\dots,D-1 \tag{41}$$

For all $x \in S^D$, there exist coefficients $(z_i)_{i\in[\![1,D-1]\!]}$ in $\mathbb{R}^{D-1}$ such that :

$$x = \bigoplus_{i=1}^{D-1} z_i \odot b_i \quad \text{where } z_i = \mathrm{ilr}(x)_i \tag{42}$$

where

$$\mathrm{ilr} : \begin{cases} S^D & \longrightarrow \mathbb{R}^{D-1} \\ x & \longmapsto V^T \log(x) \end{cases} \tag{43}$$

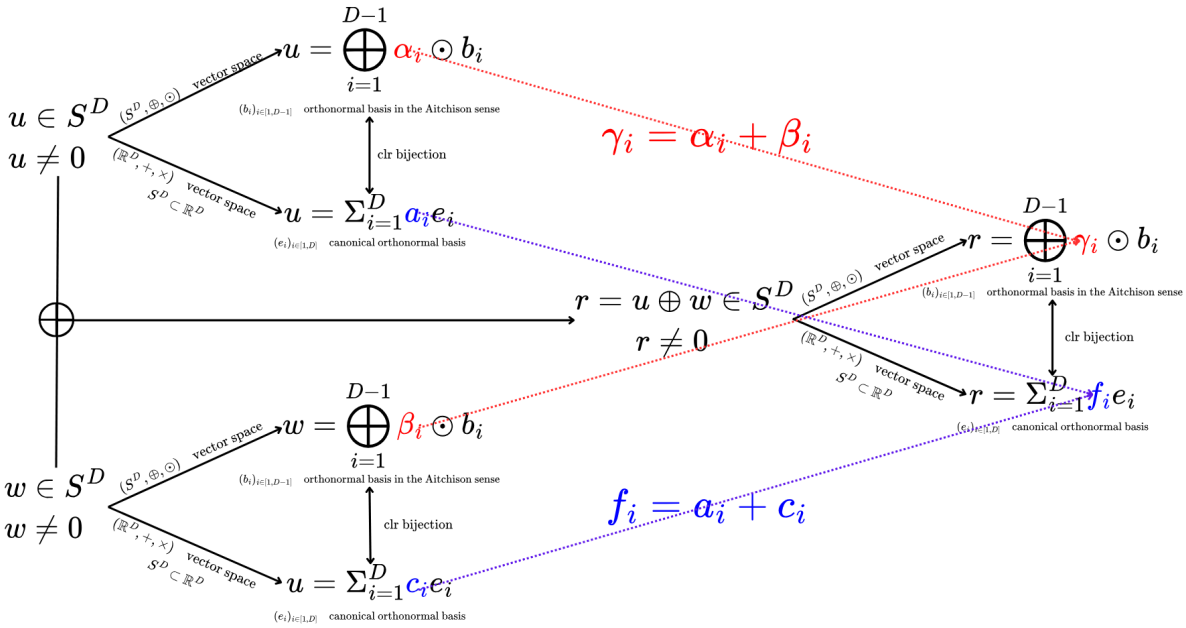

Figure 3: Diagram to sum up all the notations and the different coordinate systems.

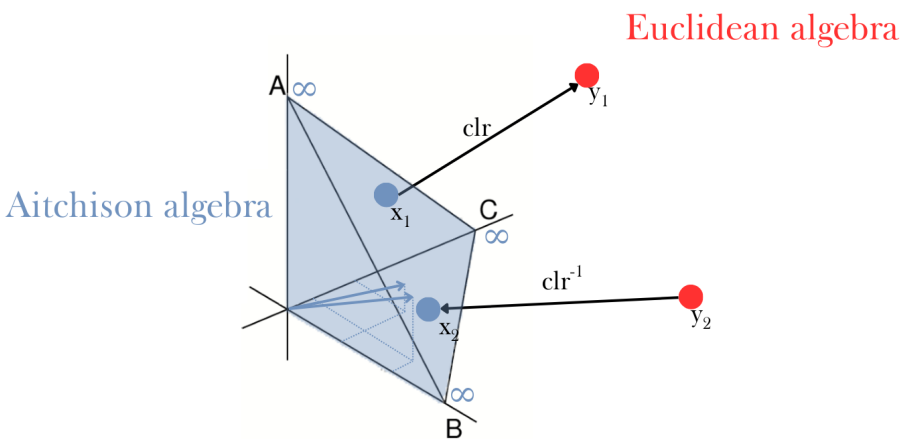

Figure 4: We have added the basis of S^3 to Figure 2 : two vectors in the Euclidean coordinate system that we determined after applying (40) and (41) for $D = 3$. The edges of the simplex correspond to Aitchison infinity.

2.14. Definition (Aitchison Norm)

For all $x \in S^D \setminus \{0\}$:

$$\|x\|_a = \sqrt{\frac{1}{2D} \sum_{i=1}^{D} \sum_{j=1}^{D} \left(\ln \frac{x_i}{x_j} \right)^2} \tag{44}$$

2.15. Definition (Aitchison Distance)

For all $(x, y) \in (S^D \setminus \{0\})^2$:

$$d_a(x, y) = \|x \ominus y\|_a = \sqrt{\frac{1}{2D} \sum_{i=1}^{D} \sum_{j=1}^{D} \left(\ln \frac{x_i}{x_j} - \ln \frac{y_i}{y_j} \right)^2} \tag{45}$$

The Aitchison distance satisfies the definition of a metric, as it is always non-negative, respects the Cauchy-Schwarz inequality, satisfies the triangle inequality, and follows Pythagoras' theorem.

3. Implementation and benchmark in Python

3.1. Usual GD in euclidean space

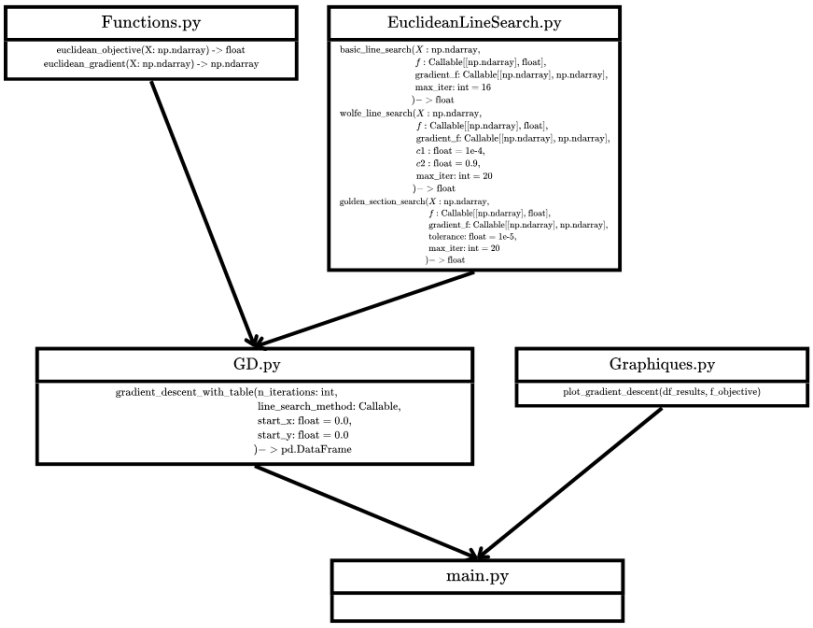

Figure 5: Module dependency diagram for the euclidean gradient descent implementation. Please check the code for further explanations.

Let's apply our implementation of euclidean GD on the following example :

$$f : \begin{cases} \mathbb{R}^2 & \longrightarrow \mathbb{R} \\ (x, y) & \longmapsto \log\left(1 + \exp\left(x^2 + y^2\right)\right) + \sqrt{x^2 + y^2 + 1} \end{cases} \tag{46}$$

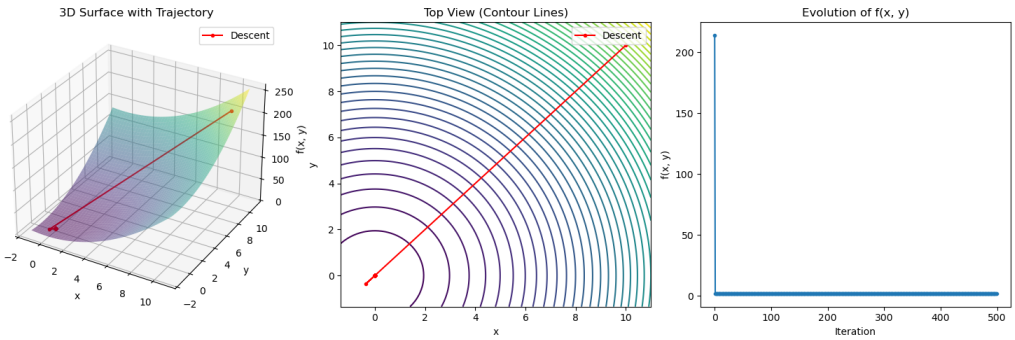

Figure 6: Euclidean GD basic_line_search.

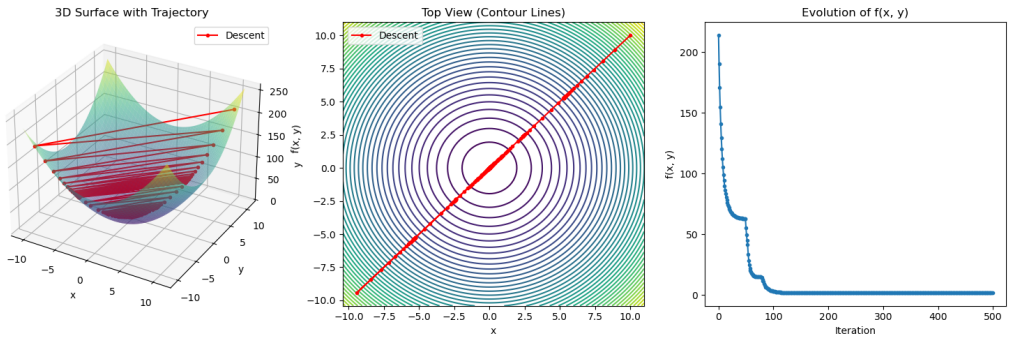

Figure 7: Euclidean GD wolfe_line_search.

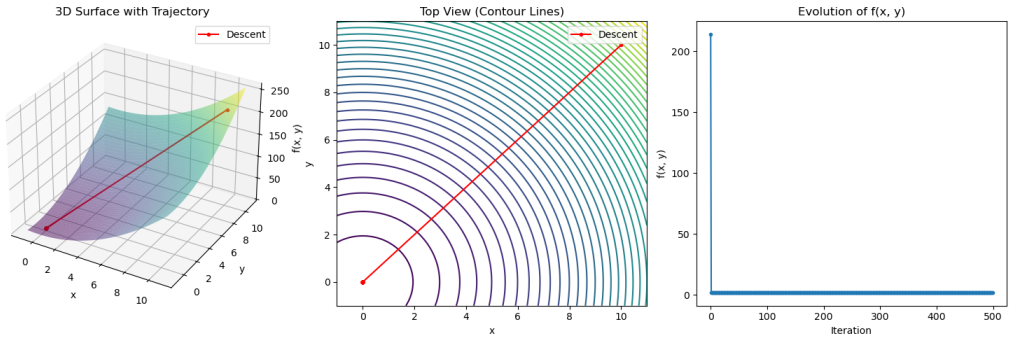

Figure 8: Euclidean GD golden_section_search.

3.2. GD in Aitchison algebra

We need to think carefully to explicitly formulate Aitchison GD. Although there is no theoretical proof of convergence in this paper as in 1.8, an empirically conclusive approach is outlined in the following diagram.

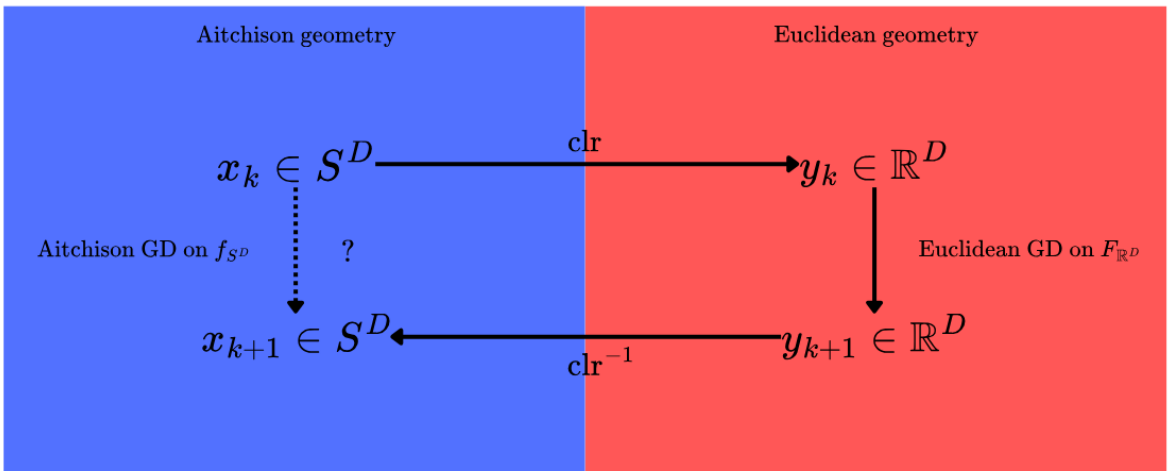

Figure 9: Aitchison GD explanation diagram. We apply the clr transformation to work in Euclidean space. Then, we perform Euclidean GD before converting back to Aitchison algebra using the inverse clr transformation in order to make sure the algorithms decreases over the iterations. This approach circumvents the difficulty of explicitly formulating the Aitchison gradient by leveraging Euclidean geometry, at the cost of establishing the equivalence between f_{S^D}, the function in the Aitchison algebra, and $F_{\mathbb{R}^D}$, the expression of f_{S^D} in Euclidean space, which is not a trivial task. In other words :

$$x_{k+1} = \mathrm{clr}^{-1}\left(\mathrm{clr}(x_k) - \alpha_k \nabla F_{\mathbb{R}^2}(\mathrm{clr}(x_k))\right) \tag{47}$$

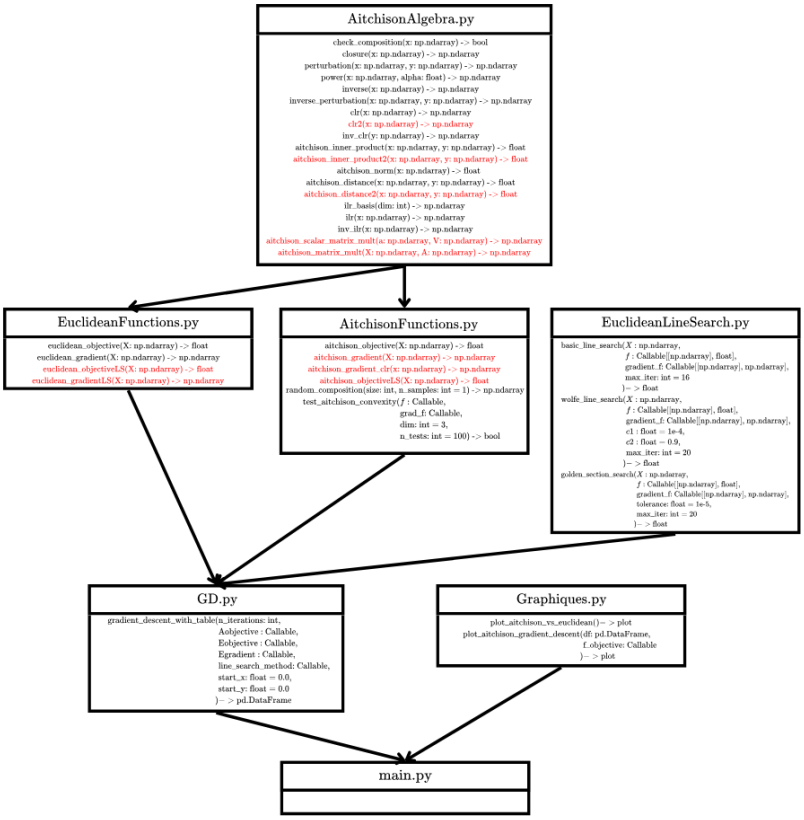

Figure 10: Module dependency diagram for the Aitchison gradient descent implementation. Please check the code for further explanations. The functions in red are unused as they will be covered later, are still in development, or under consideration.

Let's apply our implementation of Aitchison GD on the following example :

$$f_{S^2} : \begin{cases} S^2 & \longrightarrow \mathbb{R} \\ x & \longmapsto d_a(x,y)^2, \quad \text{with } y = [\frac{1}{3}, \frac{2}{3}] \end{cases} \tag{48}$$

$$F_{\mathbb{R}^2} : \begin{cases} \mathbb{R}^2 & \longrightarrow \mathbb{R} \\ x & \longmapsto \|x-y\|_2^2, \quad \text{with } y = \mathrm{clr}\left([\frac{1}{3}, \frac{2}{3}]\right) \end{cases} \tag{49}$$

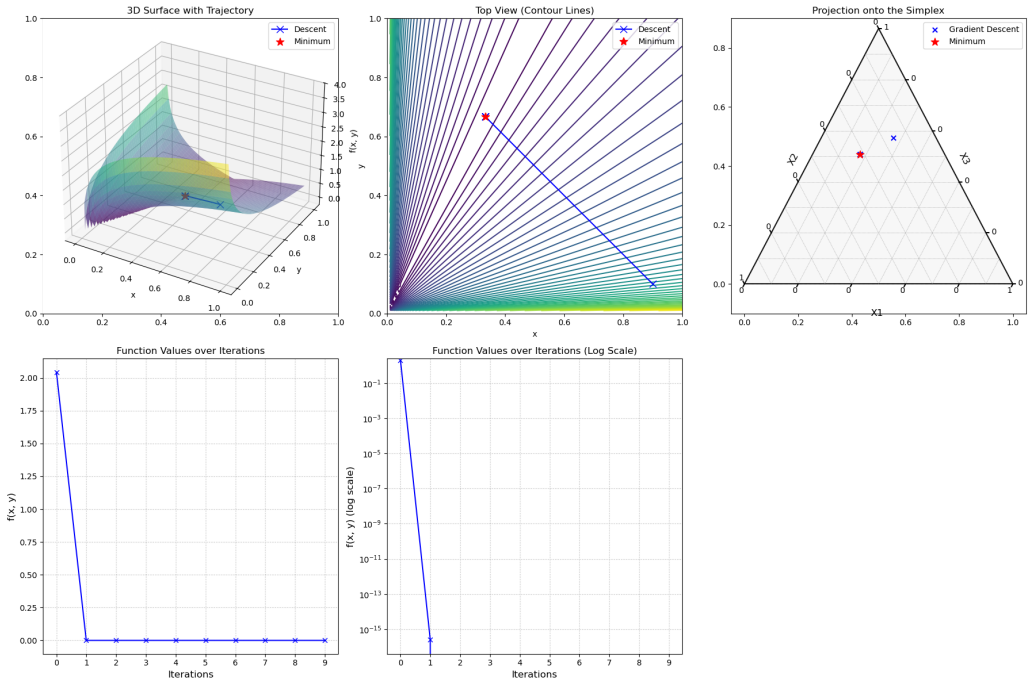

Figure 11: Projection of 10 iterations of Aitchison GD with basic line search onto a ternary diagram. The red dot is the location of y on the simplex. Consistent with convergence speed plotted in figure 3.

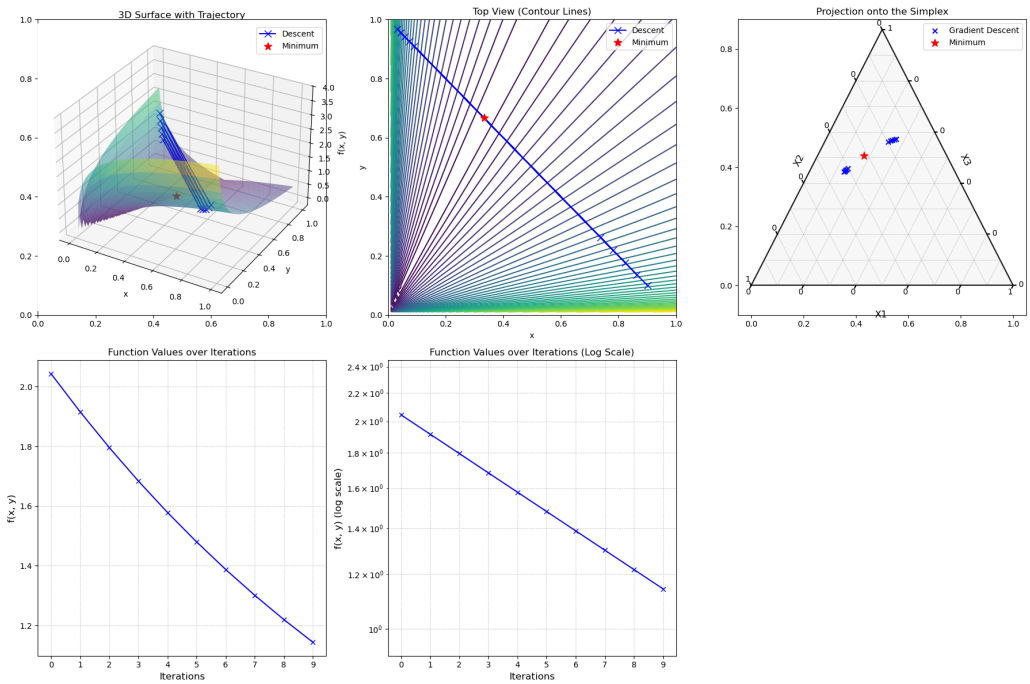

Figure 12: Projection of 10 iterations of Aitchison GD with Wolfe line search onto a ternary diagram. The red dot is the location of y on the simplex. Consistent with convergence speed plotted in figure 4.

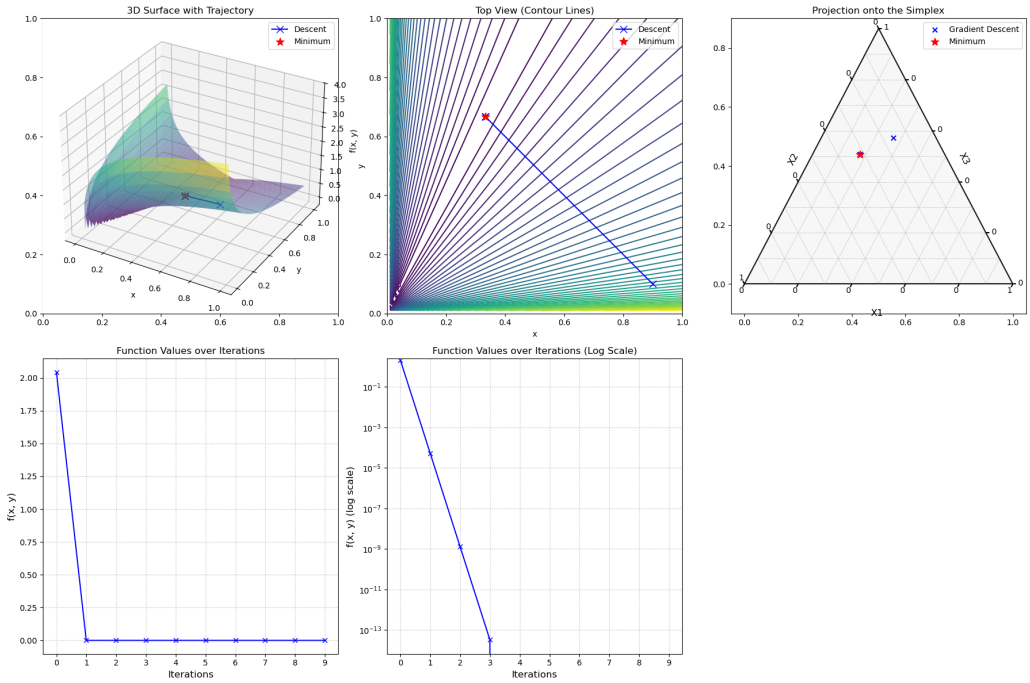

Figure 13: Projection of 10 iterations of Aitchison GD with golden section search onto a ternary diagram. The red dot is the location of y on the simplex. Consistent with convergence speed plotted in figure 5.

Once again, there's a trick to make this method work : we need to find f_{S^D} and $F_{\mathbb{R}^D}$. For norms, it's straightforward — we simply take the norms in each of the two spaces, but this is not easily generalisable. At this point of the project, we can only study certain forms of functions that can be expressed as norms.

4. Failed attempts at implementing Aitchison GD and areas of improvement

The following solutions have no reason to converge, as there is no proof. If that is the case, we should try to provide one.

4.1. Approach 1

A naive approach would be to replace Euclidean operations with their Aitchison equivalents :

$$x_{k+1} = x_k \ominus \gamma_k \odot \nabla f(x_k) \tag{50}$$

However, this does not make any sense : why would we replace only the Euclidean operations but not the gradient itself ? Therefore, we also need to consider how to adapt $\nabla_E f(x_k)$. Two approaches follow from it...

4.2. Approach 2

...We can either follow the same logic as in 3.2. but apply it only to the gradient term of the algorithm. This leads to the following algorithm proposal :

$$x_{k+1} = x_k \ominus \gamma_k \odot \mathrm{clr}^{-1} \circ \nabla_E \circ f_{S^D} \circ \mathrm{clr}(x_k) \tag{51}$$

However, this does not make much sense either because f_{S^D} might not be well-defined for any $\mathrm{clr}(x)$, as f_{S^D} is a function defined on S^D, whereas $\mathrm{clr}(x)$ belongs to Euclidean space.

One possible solution would be to replace f_{S^D} with its Euclidean equivalent $F_{\mathbb{R}^D}$. This approach would work like in 3.2., but unfortunately, there was not enough time during the project to implement it.

4.3. Approach 3

... Or redefine the gradient operator for Aitchison algebra and compute it for the function under study. Let's review the fundamentals as a refresher.

4.3.1. Definition (Euclidean basis decomposition and Euclidean directional derivative)

Let $(e_i)_{i \in [\![1,D]\!]}$ be an orthonormal basis of $\mathbb{R}^D$. Then, any vector $v \in \mathbb{R}^D$ can be uniquely decomposed in terms of this basis as follows :

$$\exists\,(v_1, \dots, v_D) \in \mathbb{R}^D, \quad v = v_1 e_1 + \cdots + v_D e_D \tag{52}$$

$$= [e_1 \mid \cdots \mid e_D] \times \begin{bmatrix} v_1 \\ \vdots \\ v_D \end{bmatrix} \tag{53}$$

$$= I_D \times \begin{bmatrix} v_1 \\ \vdots \\ v_D \end{bmatrix} \tag{54}$$

The directional derivative of a function $F_{\mathbb{R}^D}$ at $x \in \mathbb{R}^D$ in the direction $v \in \mathbb{R}^D$ is defined as follows :

$$D_v F_{\mathbb{R}^D}(x) = \langle \nabla_E F_{\mathbb{R}^D}(x), v \rangle \tag{55}$$

$$= \lim_{h \to 0} \frac{F_{\mathbb{R}^D}(x + hv) - F_{\mathbb{R}^D}(x)}{h} \tag{56}$$

4.3.2. Definition (Euclidean gradient)

Let

$$F_{\mathbb{R}^D} : \begin{cases} \mathbb{R}^D & \longrightarrow \mathbb{R} \\ x = [x_1, \dots, x_D] & \longmapsto F_{\mathbb{R}^D}(x) \end{cases} \tag{57}$$

The Euclidean gradient of $F_{\mathbb{R}^D}$ is defined as:

$$\nabla_E F_{\mathbb{R}^D} : \begin{cases} \mathbb{R} & \longrightarrow \mathbb{R}^D \\ F_{\mathbb{R}^D}(x) & \longmapsto \left[\frac{\partial F_{\mathbb{R}^D}}{\partial x_1}(x), \dots, \frac{\partial F_{\mathbb{R}^D}}{\partial x_D}(x)\right] = [D_{e_1} F_{\mathbb{R}^D}(x), \dots, D_{e_D} F_{\mathbb{R}^D}(x)] \end{cases} \tag{58}$$

4.3.3. Definition (Aitchison basis decomposition and Aitchison directional derivative)

Let $(b_i)_{i \in [\![1, D-1]\!]}$ be an orthonormal basis of S^D, its explicit expression is obtained by reading the columns of matrix V defined in 2.13. Then, any vector $v \in S^D$ can be uniquely decomposed in terms of this basis as follows :

$$\exists\, (v_1, \dots, v_D) \in \mathbb{R}^{D-1}, \quad v = v_1 \odot e_1 \oplus \cdots \oplus v_{D-1} \odot e_{D-1} \tag{59}$$

$$= \bigoplus_{i=1}^{D-1} v_i \odot e_i \tag{60}$$

The directional derivative of a function f_{S^D} at $x \in S^D$ in the direction $v \in S^D$ is defined as follows :

$$D_v f_{S^D}(x) = \langle \nabla_A f_{S^D}(x), v \rangle \tag{61}$$

$$= \lim_{h \to 0} \frac{f_{S^D}(x \oplus h \odot v) - f_{S^D}(x)}{h} \tag{62}$$

4.3.4. Definition (Aitchison gradient)

Let

$$f_{S^D} : \begin{cases} S^D & \longrightarrow \mathbb{R} \\ x = [x_1, \dots, x_{D-1}] & \longmapsto f_{S^D}(x) \end{cases} \tag{63}$$

The Aitchison gradient of f_{S^D} is defined as:

$$\nabla_A f_{S^D} : \begin{cases} \mathbb{R} & \longrightarrow \mathbb{R}^{D-1} \\ f_{S^D}(x) & \longmapsto \left[D_{b_1} f_{S^D}(x), \dots, D_{b_{D-1}} f_{S^D}(x)\right] \end{cases} \tag{64}$$

4.4. Heatmap on ternary diagram

I attempted to generate a heatmap of the function using the `mpltern` library in Python. However, the resulting plot only displayed a single spot, which was not the expected outcome. It appears that the heatmaps vary significantly depending on the distribution of the points. Unfortunately, this approach did not yield the desired results, as the generated plot did not provide meaningful insights. This aspect of the analysis was less successful than anticipated.

5. Conclusion and research perspectives

This project has primarily focused on the theoretical exploration of Aitchison's algebra, highlighting its mathematical foundations, its conceptual potential and a quick application to GD. Although confined to a theoretical framework, our work opens promising perspectives for future developments.

5.1. Additional functions for testing Aitchison GD and Aitchison Least Squares

Let $x \in S^D$ and A be a matrix in $\mathcal{M}_D(\mathbb{R})$. The matrix product $y = x \otimes A$ of these two matrices is defined as follows:

$$y = \mathcal{C}\left[\prod_{j=1}^{D} x_j^{a_{j1}}, \prod_{j=1}^{D} x_j^{a_{j2}}, \ldots, \prod_{j=1}^{D} x_j^{a_{jD}}\right] \tag{65}$$

We could apply these following functions to the Aitchison GD :

$$f_{S^2} : \begin{cases} S^2 & \longrightarrow \mathbb{R} \\ x & \longmapsto \|x \otimes A \ominus y\|_a^2 \quad \text{with } y = [\frac{1}{3}, \frac{2}{3}] \end{cases} \tag{66}$$

$$F_{\mathbb{R}^2} : \begin{cases} \mathbb{R}^2 & \longrightarrow \mathbb{R} \\ x & \longmapsto \|xA - y\|_2^2 \quad \text{with } y = \operatorname{clr}\left(\left[\frac{1}{3}, \frac{2}{3}\right]\right) \end{cases} \tag{67}$$

We have thus defined a form of Aitchison Least Squares problem : The Aitchison GD might exhibit different dynamics in terms of complexity and the number of iterations, depending on certain properties of A. More generally, we could begin to define Aitchison versions of various Machine Learning techniques to adapt Euclidean geometry methods to Aitchison algebra.

5.2. Image processing

In the context of image processing, the concepts of Aitchison geometry could provide a new perspective on traditional methods. Typically, images are represented using pixel values in a color space such as RGB, where each pixel contains a proportion of red, green, and blue values. These values can be interpreted as compositional data, where each pixel's color is a composition of its RGB components.
In standard image processing methods, Euclidean geometry is often used to measure distances and similarities between image pixels. However, by applying the Aitchison geometry, we can treat the RGB values as compositions and analyse them using the algebraic structures of compositional data.
This could lead to more accurate models, especially when dealing with proportions, and might improve tasks such as image segmentation, classification, or denoising. The challenge lies in adapting traditional image processing techniques to this new framework, optimising algorithms to exploit the properties of Aitchison geometry, and reducing computational complexity while enhancing performance. Exploring this approach could potentially open up new avenues for more robust and efficient image processing methods.

References

[1] Pawlowsky-Glahn, V., Egozcue, J. J., Tolosana-Delgado, R. (2007). *Lecture Notes on Compositional Data Analysis.*

[2] Pawlowsky-Glahn, V. (2007). *The Aitchison geometry of the simplex and the statistical analysis of compositional data.* Department of Computer Science and Applied Mathematics, University of Girona, Spain. Facultad de Ciencias – Departamento de Matemáticas, Universidad de los Andes, Bogota, Colombia, March 15, 2007.

[3] Pawlowsky-Glahn, V., Egozcue, J. J., Tolosana-Delgado, R. (2011). *Lecture Notes on Compositional Data Analysis.* University of Girona, Spain; Technical University of Catalonia, Spain.

[4] Mildenberger, T. (2016). *Compositional Data.* Datalab Seminar, Institut für Datenanalyse und Prozessdesign, School of Engineering, Zürcher Hochschule für Angewandte Wissenschaften.

[5] Beck, A. (2017). *First-Order Methods in Optimization.*

[6] Erb, I., Ay, N. (2020). *The Information-Geometric Perspective of Compositional Data Analysis.* arXiv:2005.11510v3.

[7] Ang, A. (2024). *Quick slide on Non-standard NMF.* CVX seminar, CO, U. Waterloo, ECS, U. Southampton, UK.